

06. 자료 구조와 알고리즘

10. 트리

학습목표

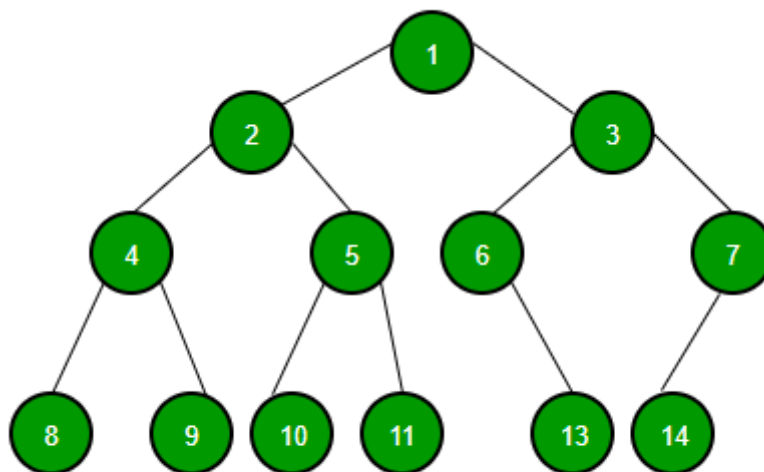
- 트리가 무엇인지 알 수 있다.
- 트리를 적재적소에 활용할 수 있다.

들어가며

트리(Tree)는 **그래프의 일종으로 계층 구조를 가지고, 방향성이 있으며, 순환(Cycle)이 불가능한 그래프**다.* 그래서 시작점이 되는 루트 노드(Root Node)라는 것이 존재하고, 루트에서 아래로 뻗어가는 형태**이며, 두 노드 사이의 경로는 오직 하나다. 이번 시간에는 트리와 관련 컬렉션 사용을 알아보도록 하자.

* DAG(Directed Acyclic Graph)다.

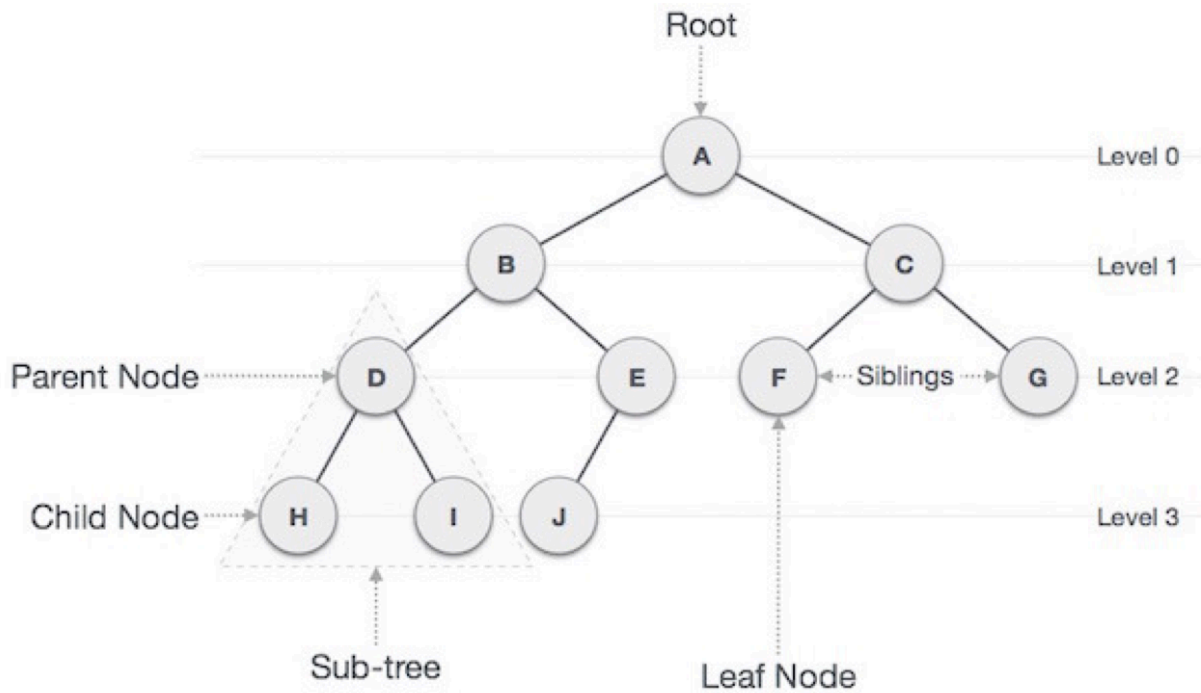
** 후술하겠지만 부모에서 자식으로의 방향성을 가진다고 볼 수 있다. 물론 실제 노드를 구현할 때는 부모에 대한 참조를 갖고 있기도 한다.



트리의 모습

용어

트리에는 여러 가지 용어가 있다. 아래의 그림을 참고하자.



이 외의 용어는 아래와 같다.

- 조상(Ancessor) 노드 : 한 노드에서 간선을 따라 루트 노드까지 이르는 경로에 있는 모든 노드
- 자손(Descendant) 노드 : 조상 노드의 반대
- 차수(Degree) : 한 노드가 가지는 서브 트리의 수. 노드의 차수 중 최댓값을 트리의 차수라 한다.
- 내부(Internal) 노드 : 단말 노드를 제외한 나머지 노드
- 포레스트(Forest) : 트리의 집합

순회

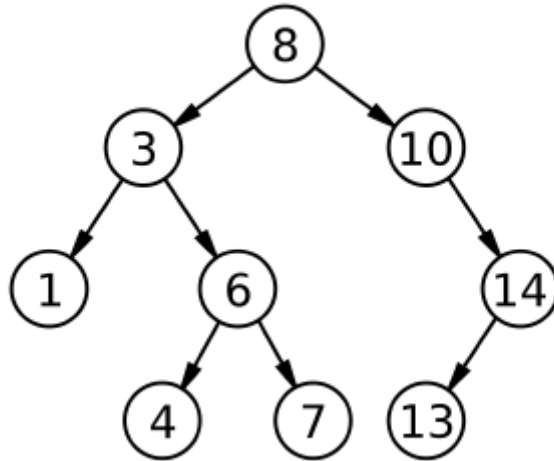
트리에 대한 순회는 전위 순회(Pre-order Traversal) / 중위 순회(In-order Traversal) / 후위 순회(Post-order Traversal) / 레벨 순회(Level-Order Traversal)가 있다. 위의 이미지를 토대로 순회 순서를 나타내면 아래와 같다.

- 전위 순회 : A - B - D - H - I - E - J - C - F - G
- 중위 순회 : H - D - I - B - J - E - A - F - C - G
- 후위 순회 : H - I - D - J - E - B - F - G - C - A
- 레벨 순회 : A - B - C - D - E - F - G - H - I - J

이진 검색 트리

이진 검색 트리(Binary Search Tree)는 한 노드를 기준으로 왼쪽 서브 트리는 모두 그 노드보다 작은 값을 가지고 있으며, 오른쪽 서브 트리는 모두 그 노드보다 큰 값을 가지고 있는 **이진 트리***(Binary tree)이다. 수식으로 표현하면 왼쪽 자식 노드의 값 $\leq$ 부모 노드의 값 $\leq$ 오른쪽 자식 노드의 값이 된다. 이때문에 이진 검색 트리를 중위 순회하면 정렬된 데이터를 얻을 수 있다.

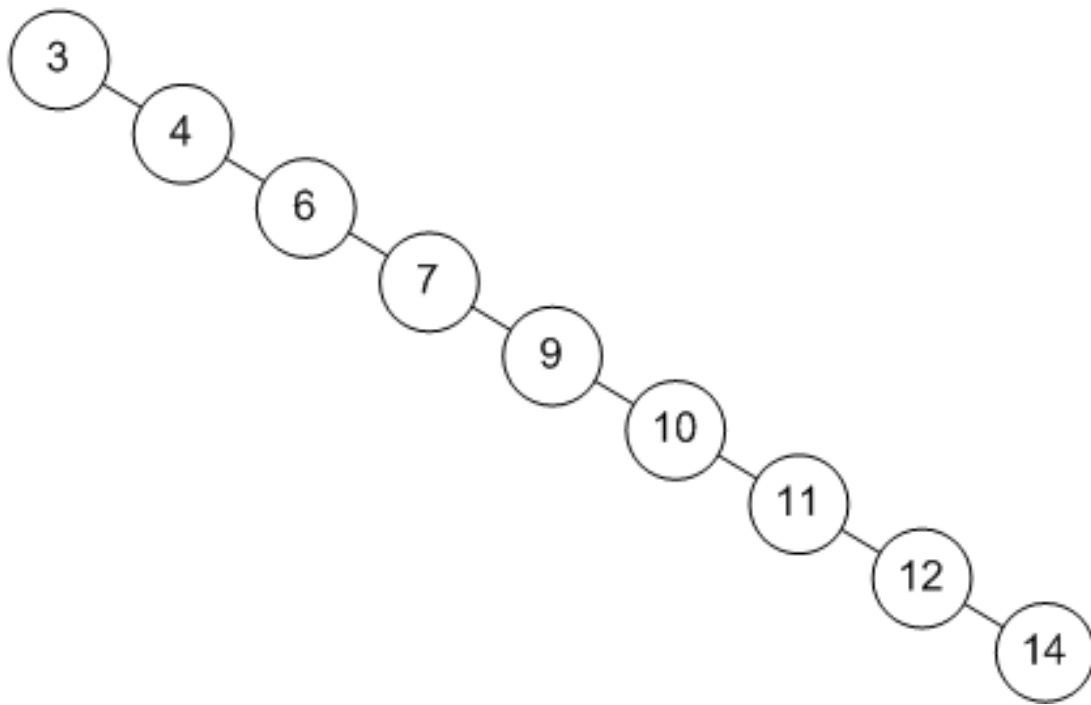
*이진 트리 : 트리의 차수가 2 이하인 트리



이진 검색 트리

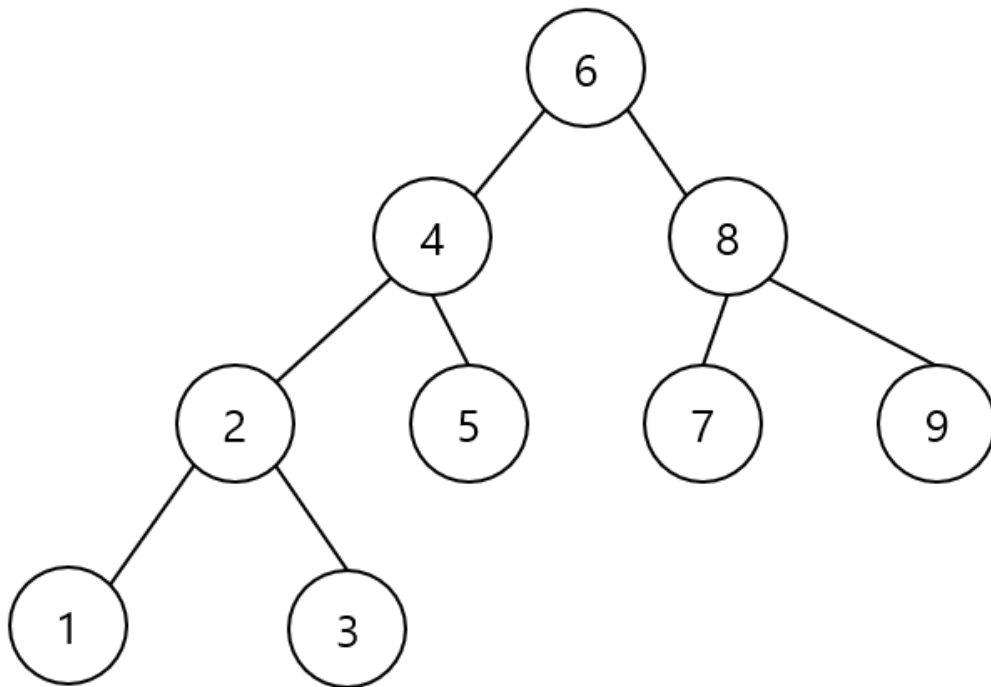
트리의 균형

이진 검색 트리는 데이터의 삽입 순서에 따라 형태가 달라지는데, 이에 따라 **트리의 균형도**가 결정된다. 트리의 균형도는 **왼쪽 서브 트리의 높이와 오른쪽 서브 트리의 높이의 차**를 말하는데, **균형도의 절댓값이 2이상이면 트리가 편향되었다고 표현한다**. 트리의 균형도는 트리의 연산에 영향을 미친다. 트리가 균형잡혀 있다면 모든 연산이 $O(\log N)$ 이지만, 편향되어 있다면 $O(N)$ 에 가까워진다.



오른쪽으로 편향된 트리다. 검색, 삽입, 삭제, 읽기 연산이 모두 $O(N)$ 이다.

그래서 어떻게 하면 트리의 균형을 유지할 수 있을까를 고민했고, 이에 따라 자가 균형 이진 검색 트리(Self-balancing Binary Search Tree)가 생겨나게 되었다. 여기에는 AVL 트리와 레드 블랙 트리가 존재한다.



균형 잡힌 이진 검색 트리다. 검색, 삽입, 삭제, 읽기 연산이 모두 $O(\log N)$ 이다.

사용법

C#에서는 [SortedSet](#)과 [SortedDictionary](#)로 구현되어 있으며, 둘 다 레드 블랙 트리다. 먼저 SortedSet부터 알아보자.

```

SortedSet<int> set = new SortedSet<int>();

// # 삽입
// Add()로 데이터를 삽입하며, 반환값으로 삽입의 여부를 알 수 있다.
Console.WriteLine(set.Add(1)); // True
Console.WriteLine(set.Add(2)); // True
Console.WriteLine(set.Add(3)); // True

// 중복된 데이터를 허용하지 않는다.
Console.WriteLine(set.Add(1)); // False

// # 읽기
// Max, Min으로 최댓값과 최솟값을 알 수 있다.
Console.WriteLine(set.Min); // 1
Console.WriteLine(set.Max); // 3

// # 검색
// Contains()로 데이터의 존재 여부를 검색할 수 있다.
Console.WriteLine(set.Contains(1)); // True
Console.WriteLine(set.Contains(4)); // False

// # 삭제
// Remove()로 데이터를 삭제할 수 있으며, 반환값으로 삭제의 여부를 알 수 있다.
Console.WriteLine(set.Remove(1)); // True
Console.WriteLine(set.Remove(4)); // False

```

그 다음은 SortedDictionary이다. SortedSet과는 다르게 데이터를 저장할 때 키와 값의 쌍으로 저장한다. 비교의 경우 키에 대해서 연산한다.

```

SortedDictionary<string, string> openWith = new SortedDictionary<string, string>();

// # 삽입
// Add()로 데이터를 삽입한다.
openWith.Add("txt", "notepad.exe");
openWith.Add("bmp", "paint.exe");

// 중복된 키를 허용하지 않는다.
openWith.Add("txt", "vscode.exe"); // 오류 발생함.

// 인덱서로도 삽입할 수 있다.
openWith["html"] = "chrome.exe";

// # 읽기
// 인덱서로 데이터를 읽을 수 있다.
Console.WriteLine(openWith["txt"]); // "notepad.exe"
Console.WriteLine(openWith["bmp"]); // "paint.exe"

// # 검색
// ContainsKey()로 특정 키가 존재하는지 알 수 있다.
Console.WriteLine(openWith.ContainsKey("txt")); // True
Console.WriteLine(openWith.ContainsKey("html")); // False

// ContainsValue()로 특정 값이 존재하는지 알 수 있다.
// ! 선형 검색이다.
Console.WriteLine(openWith.ContainsValue("notepad.exe")); // True
Console.WriteLine(openWith.ContainsValue("rider.exe")); // False

// # 삭제
// Remove()로 데이터를 삭제할 수 있으며, 반환값으로 삭제의 여부를 알 수 있다.
Console.WriteLine(openWith.Remove("txt")); // True
Console.WriteLine(openWith.Remove("cs")); // False

```

더 나아가기

1. 일반적인 그래프와 비교했을 때, 트리가 가지는 가장 큰 차이점(특징) 2가지는 무엇인가요?
2. 트리와 관련된 용어에 대해서 정리해주세요.
3. 이진 검색 트리에서 중위 순회를 수행했을 때 얻을 수 있는 결과물의 특징은 무엇인가요?
4. 트리의 높이를 구하는 알고리즘을 설계할 때, 어떤 순회 방식의 아이디어를 활용하는 것이 가장 적합한가요?
5. AVL 트리는 삽입과 삭제 시 어떻게 균형을 맞추는지 이 자료([1](#), [2](#))를 통해 알아보세요.
6. 레드 블랙 트리는 삽입과 삭제 시 어떻게 균형을 맞추는지 이 자료([1](#), [2](#), [3](#))을 통해 알아보세요.
7. 이진 검색 트리를 구현해봅시다. 아래 코드를 완성해주세요.

```
class Node
{
    public int Data { get; set; }
    public Node Left { get; set; } // 왼쪽 자식
    public Node Right { get; set; } // 오른쪽 자식
}

class BinarySearchTree
{
    // 루트 노드는 초기화 되어 있다고 가정한다.
    private Node _root;

    public void Preorder()
    {
        // TODO
        // 전위 순회를 구현하세요.
    }

    public void Inorder()
    {
        // TODO
        // 중위 순회를 구현하세요.
    }
}
```

```

public void Postorder()
{
    // TODO
    // 후위 순회를 구현하세요.
}

public void Levelorder()
{
    // TODO
    // 레벨 순회를 구현하세요.
}

public int Height
{
    get
    {
        // TODO
        // 트리의 높이를 구하는 코드를 작성하세요.
    }
}

public bool Contains(int value)
{
    // TODO
    // Contains를 완성하세요.
    // 트리에 value가 있다면 true, 없다면 false입니다.
}
}

```

8. [SortedSet](#)의 메소드 및 프로퍼티를 정리해보세요.
9. [SortedDictionary](#)의 메소드 및 프로퍼티를 정리해보세요.
10. [SortedList](#)는 [SortedDictionary](#)랑 어떤 [차이](#)가 있을까요?
11. 아래 문제를 풀어봅시다.
 - a. [11725번: 트리의 부모 찾기](#)

- b. [1991번: 트리 순회](#)
- c. [1967번: 트리의 지름](#)
- d. [1167번: 트리의 지름](#)
- e. [5639번: 이진 검색 트리](#)
- f. [1992번: 쿼드트리](#)

참고자료

- [C로 배우는 쉬운 자료구조](#)
- [누구나 자료 구조와 알고리즘](#)
- [SortedSet<T> Class \(System.Collections.Generic\) | Microsoft Learn](#)
- [SortedDictionary<TKey,TValue> Class \(System.Collections.Generic\) | Microsoft Learn](#)
- [Binary Search Tree, AVL Tree - VisuAlgo](#)
- [Binary Search Tree - GeeksforGeeks](#)